

Trabajo de Laboratorio 03: Comportamiento de la memoria virtual y paralelismo a nivel de instrucción

Cesar Lengua Malaga

29 de marzo de 2026

Resumen

En este informe se presentan dos ejemplos sencillos relacionados con modificaciones modernas del modelo de Von Neumann: la memoria virtual y el paralelismo a nivel de instrucción. El objetivo no es reproducir toda la complejidad interna del hardware y del sistema operativo, sino mostrar de manera básica cómo funcionan estas ideas y por qué representan mejoras sobre el modelo clásico. Los ejemplos fueron implementados en C++ y permiten observar de forma simple el uso de páginas de memoria y la diferencia entre instrucciones dependientes e instrucciones más independientes.

1. Introducción

El modelo de Von Neumann describe una computadora organizada alrededor de una unidad de procesamiento, una memoria principal y un programa almacenado. En este modelo clásico, las instrucciones y los datos comparten la memoria, y la ejecución del programa se entiende principalmente como una secuencia ordenada de pasos.

Con el avance de la arquitectura de computadores, aparecieron mejoras para superar algunas limitaciones del modelo original. Entre ellas se encuentran la memoria virtual, que permite administrar mejor la memoria disponible, y el paralelismo a nivel de instrucción, que permite aprovechar mejor los recursos internos del procesador.

En este trabajo se implementan ejemplos básicos de ambas ideas para ilustrar su funcionamiento y facilitar su explicación.

2. Objetivos

- Mostrar un ejemplo simple del uso de memoria virtual mediante páginas.
- Mostrar un ejemplo simple de paralelismo a nivel de instrucción mediante operaciones dependientes e independientes.
- Relacionar ambos ejemplos con modificaciones modernas del modelo de Von Neumann.
- Presentar una explicación clara y sencilla que pueda utilizarse en la exposición del tema.

3. Marco teórico

3.1. Modelo de Von Neumann

El modelo de Von Neumann se basa en tres ideas principales:

- Existe una memoria principal donde se almacenan tanto los datos como las instrucciones.

- Existe una unidad central de procesamiento que interpreta y ejecuta las instrucciones.
- La ejecución del programa sigue una secuencia de pasos, normalmente descrita como buscar, decodificar y ejecutar.

Este modelo sigue siendo la base de la mayoría de computadores actuales. Sin embargo, los sistemas modernos incorporan modificaciones para mejorar el rendimiento y el manejo de recursos.

3.2. Memoria virtual

La memoria virtual es una técnica que permite que cada proceso trabaje con un espacio de direcciones propio, llamado espacio de direcciones virtuales. El programa no accede directamente a direcciones físicas de RAM, sino a direcciones virtuales que luego son traducidas por el sistema operativo y por el hardware.

La memoria virtual se organiza normalmente en páginas. Cuando un programa necesita una página, el sistema puede cargarla en memoria física si aún no se encuentra disponible. Esto permite ejecutar programas grandes, aislar procesos y administrar de mejor manera la memoria.

3.3. Paralelismo a nivel de instrucción

El paralelismo a nivel de instrucción, también llamado ILP por sus siglas en inglés, consiste en aprovechar que algunas instrucciones de un programa no dependen entre sí. Cuando eso ocurre, el procesador puede solapar o ejecutar varias etapas al mismo tiempo dentro de su pipeline.

Si las instrucciones son muy dependientes, el procesador debe esperar el resultado de una para continuar con la siguiente. Si las instrucciones son más independientes, el procesador puede trabajar de forma más eficiente.

4. Descripción de las implementaciones

4.1. Ejemplo 1: memoria virtual

En el archivo `memoria_virtual.cpp` se reserva un arreglo grande usando un `vector<int>`. Luego se calcula un tamaño aproximado de página de 4 KB y se accede al primer entero de varias páginas consecutivas.

Para cada página, el programa muestra:

- el número de página,
- el índice dentro del arreglo,
- la dirección observada por el programa,
- el valor almacenado.

Este ejemplo sirve para explicar que el programa trabaja sobre direcciones virtuales y que la memoria se organiza en páginas. Aunque el código no implementa manualmente la traducción de direcciones, sí permite representar la idea central de la memoria virtual de manera didáctica.

Una salida de ejemplo del programa es la siguiente:

```
=== Ejemplo básico de memoria virtual ===
Tamaño aproximado de página: 4096 bytes
Enteros por página: 1024
Páginas reservadas: 8
```

Dirección virtual inicial del arreglo: 0x68ef70

Accediendo al primer entero de cada página:

Página 0 -> índice 0 -> dirección 0x68ef70 -> valor 1

Página 1 -> índice 1024 -> dirección 0x68ff70 -> valor 2

Página 2 -> índice 2048 -> dirección 0x690f70 -> valor 3

Página 3 -> índice 3072 -> dirección 0x691f70 -> valor 4

Esta salida permite observar que el programa accede a posiciones separadas por el tamaño de una página y ayuda a explicar la organización paginada de la memoria.

4.2. Ejemplo 2: paralelismo a nivel de instrucción

En el archivo `paralelismo.cpp` se comparan dos formas de sumar un arreglo:

1. Una suma con un solo acumulador.
2. Una suma con dos acumuladores.

En el primer caso, cada operación depende del resultado anterior porque siempre se actualiza la misma variable acumuladora. En el segundo caso, el trabajo se reparte entre dos acumuladores, lo que introduce más independencia entre instrucciones.

La idea del ejemplo es mostrar que el procesador moderno puede aprovechar mejor esta segunda forma, ya que tiene más oportunidades de ejecutar instrucciones en paralelo dentro del pipeline.

Una salida de ejemplo del programa es la siguiente:

=== Paralelismo a nivel de instrucción (ILP) ===

Caso 1: instrucciones dependientes

Resultado: 10000000

Tiempo: 41462 microsegundos

Caso 2: instrucciones más independientes

Resultado: 10000000

Tiempo: 30572 microsegundos

En este caso se observa que ambas versiones producen el mismo resultado, pero la segunda puede ejecutarse más rápido porque ofrece mayor independencia entre instrucciones.

5. Explicación paso a paso

5.1. Memoria virtual

El programa reserva memoria para varias páginas de enteros. Después accede al primer elemento de cada página. Este acceso representa la idea de que la memoria se divide en bloques llamados páginas.

La explicación conceptual es la siguiente:

- El programa solicita memoria.
- El sistema entrega direcciones virtuales.

- La memoria virtual se organiza por páginas.
- El sistema operativo y la unidad de manejo de memoria traducen las direcciones virtuales a direcciones físicas.

Por lo tanto, el ejemplo muestra una característica moderna del modelo de Von Neumann: la memoria ya no se entiende solo como un bloque físico lineal visible para el programa, sino como un espacio virtual administrado de manera más flexible.

5.2. Paralelismo a nivel de instrucción

El segundo programa realiza dos versiones de una suma:

- En la primera versión, la variable acumuladora genera una cadena de dependencias.
- En la segunda versión, se usan dos acumuladores y las operaciones son más independientes.

Cuando las instrucciones son dependientes, el procesador tiene menos libertad para adelantarlas. Cuando son independientes, el procesador puede aprovechar mejor sus unidades internas y su pipeline.

Este ejemplo muestra una modificación moderna del modelo de Von Neumann porque el modelo clásico suele presentarse como una secuencia estrictamente ordenada de instrucciones, mientras que los procesadores actuales buscan extraer trabajo paralelo incluso dentro de un solo hilo de ejecución.

6. Resultados observados

En la ejecución de prueba se observan dos aspectos importantes:

- En memoria virtual, el programa accede a posiciones separadas por el tamaño aproximado de una página, lo cual ayuda a visualizar la organización paginada de la memoria.
- En paralelismo a nivel de instrucción, la versión con dos acumuladores suele tardar menos tiempo que la versión con un solo acumulador, porque ofrece mayor independencia entre instrucciones.

Estos resultados son coherentes con la teoría y refuerzan la explicación conceptual de ambas modificaciones.

7. Conclusiones

El modelo de Von Neumann sigue siendo la base conceptual de la computadora moderna, pero ha sido ampliado con técnicas que mejoran su funcionamiento. La memoria virtual permite administrar la memoria de forma más flexible mediante direcciones virtuales y páginas. El paralelismo a nivel de instrucción permite que el procesador aproveche mejor su hardware interno cuando existen instrucciones independientes.

8. Código

Los códigos se encuentran en el siguiente link: <https://github.com/rxsh/Parallel/tree/main/Laboratorio%2003>